# Capstone Project Design Document

## Phase 1 – Requirements and Scope

### 1. Project overview

**Event-Driven Order Processing System** is a Spring Boot application composed of two services:

- **Order Service** – accepts orders, stores them and updates their status.
- **Fulfilment Service** – consumes order events, processes fulfilment and publishes the outcome.

The services communicate asynchronously through Apache Kafka. Each service uses an H2 in-memory database to keep the project lightweight and allow the capstone to focus on clean code, testing and event-driven architecture.

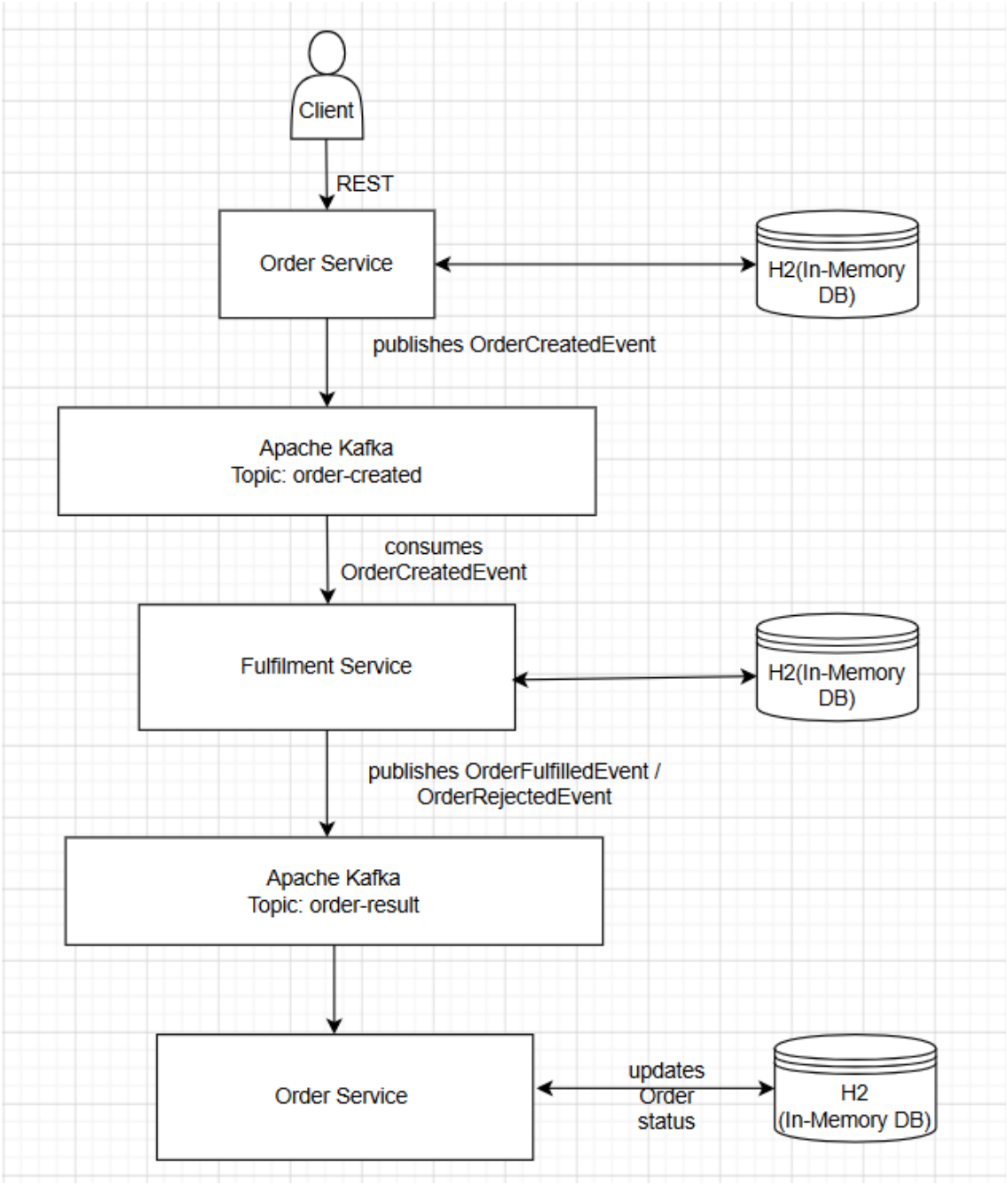
### 2. Figure 1 – High-Level Event-Driven Order Processing Flow

Figure 1 illustrates the high-level event-driven architecture of the proposed system. The Order Service receives synchronous REST requests from the client, validates and stores the order in its own H2 in-memory database before publishing an OrderCreatedEvent to Apache Kafka. The Fulfilment Service consumes the event, processes the order and stores the fulfilment result in its own H2 in-memory database before publishing either an OrderFulfilledEvent or an OrderRejectedEvent back to Kafka.

The Order Service then consumes the resulting event and updates the status of the original order in its own database. Although the diagram repeats the Order Service

and its database to illustrate the sequence of processing, these represent the same service and database, not additional components. Separate databases were chosen to reflect the microservice principle that each service owns and manages its own data independently.

**High-Level Event-Driven Order Processing Flow Diagram**

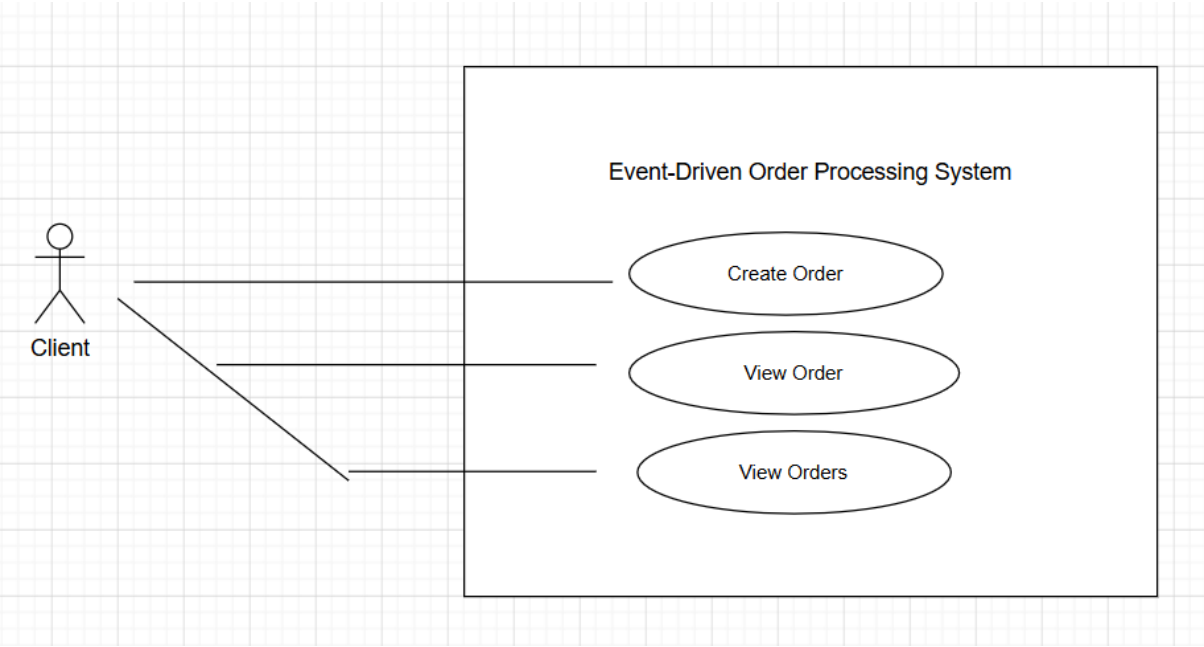



H2 in-memory databases were selected to simplify development and testing while

allowing the project to focus on Spring Boot, event-driven communication, clean object-oriented design and automated testing.

### 3. Figure 2 – UML Use Case Diagram

Figure 2 illustrates the interactions between the external client and the Event-Driven Order Processing System. The client can create a new order, retrieve an individual order and view all orders through the system's REST API. Internal activities such as Kafka messaging, fulfilment processing and database operations are intentionally omitted because they represent implementation details rather than user interactions. This high-level view defines the system's functional requirements from the user's perspective and establishes the scope for the architectural and detailed design presented in the following sections.




### 4. Figure 3 – UML Class Diagram

Figure 3 presents the detailed UML class diagram for the Event-Driven Order Processing System.

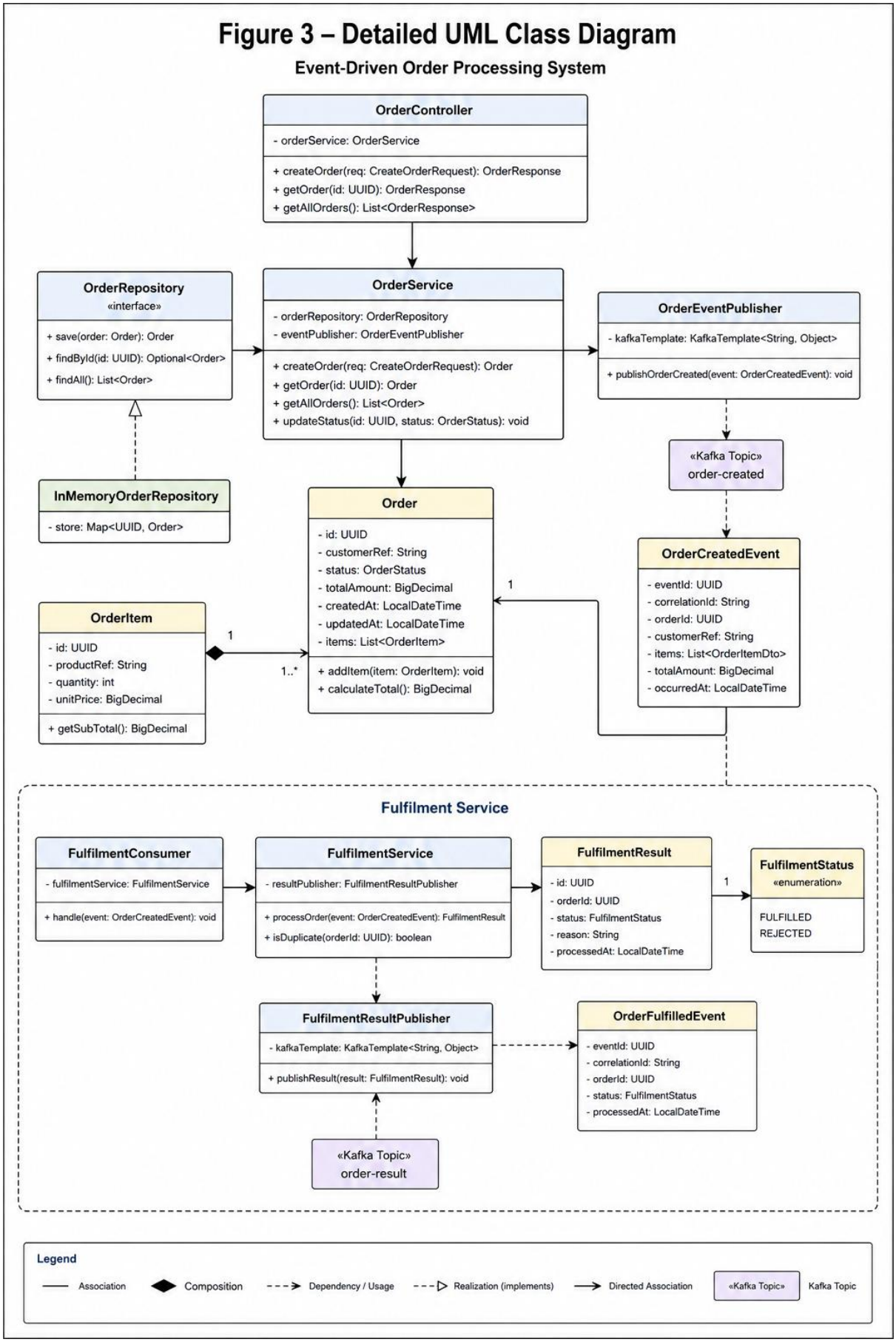


Figure 3 – Detailed UML Class Diagram
Event-Driven Order Processing System
OrderController
- orderService: OrderService
+ createOrder(req: CreateOrderRequest): OrderResponse
+ getOrder(id: UUID): OrderResponse
+ getAllOrders(): List<OrderResponse>
OrderRepository
«interface»
+ save(order: Order): Order
+ findById(id: UUID): Optional<Order>
+ findAll(): List<Order>
OrderService
- orderRepository: OrderRepository
- eventPublisher: OrderEventPublisher
+ createOrder(req: CreateOrderRequest): Order
+ getOrder(id: UUID): Order
+ getAllOrders(): List<Order>
+ updateStatus(id: UUID, status: OrderStatus): void
OrderEventPublisher
- kafkaTemplate: KafkaTemplate<String, Object>
+ publishOrderCreated(event: OrderCreatedEvent): void
«Kafka Topic»
order-created
InMemoryOrderRepository
- store: Map<UUID, Order>
Order
- id: UUID
- customerRef: String
- status: OrderStatus
- totalAmount: BigDecimal
- createdAt: LocalDateTime
- updatedAt: LocalDateTime
- items: List<OrderItem>
+ addItem(item: OrderItem): void
+ calculateTotal(): BigDecimal
OrderCreatedEvent
- eventId: UUID
- correlationId: String
- orderId: UUID
- customerRef: String
- items: List<OrderItemDto>
- totalAmount: BigDecimal
- occurredAt: LocalDateTime
OrderItem
- id: UUID
- productRef: String
- quantity: int
- unitPrice: BigDecimal
+ getSubTotal(): BigDecimal
1
1..*
1
Fulfilment Service
FulfilmentConsumer
- fulfilmentService: FulfilmentService
+ handle(event: OrderCreatedEvent): void
FulfilmentService
- resultPublisher: FulfilmentResultPublisher
+ processOrder(event: OrderCreatedEvent): FulfilmentResult
+ isDuplicate(orderId: UUID): boolean
FulfilmentResult
- id: UUID
- orderId: UUID
- status: FulfilmentStatus
- reason: String
- processedAt: LocalDateTime
1
FulfilmentStatus
«enumeration»
FULFILLED
REJECTED
FulfilmentResultPublisher
- kafkaTemplate: KafkaTemplate<String, Object>
+ publishResult(result: FulfilmentResult): void
OrderFulfilledEvent
- eventId: UUID
- correlationId: String
- orderId: UUID
- status: FulfilmentStatus
- processedAt: LocalDateTime
«Kafka Topic»
order-result
Legend
Association
Composition
Dependency / Usage
Realization (implements)
Directed Association
«Kafka Topic»
Kafka Topic

The UML class diagram demonstrates how the system applies object-oriented design principles to separate responsibilities across controllers, services, repositories and domain models. It also illustrates how event publishers and consumers integrate with the application and domain layers to support asynchronous communication while maintaining loose coupling between the Order Service and Fulfilment Service.

## 5. Functional requirements

### FR1 – Create an order

The client shall be able to submit a new order through a REST endpoint.

The request shall contain:

- customer reference
- one or more order items
- product reference
- quantity
- unit price

### FR2 – Validate an order

The Order Service shall reject an order when:

- the customer reference is missing
- no order items are supplied
- quantity is zero or negative
- unit price is zero or negative

- required fields are blank

**FR3 – Store an order**

The Order Service shall store a valid order in its H2 database with an initial status of:

PENDING

**FR4 – Publish an order-created event**

After the order has been stored, the Order Service shall publish an OrderCreatedEvent to the Kafka topic:

order-created

The event shall contain at least:

- event ID
- order ID
- customer reference
- order items
- total value
- event timestamp
- correlation ID

**FR5 – Consume the order-created event**

The Fulfilment Service shall consume OrderCreatedEvent messages from the order-created topic.

**FR6 – Process fulfilment**

The Fulfilment Service shall decide whether the order can be fulfilled.

For the first version, the decision may be based on a simple rule, such as:

total order quantity must not exceed 20 items

This keeps the logic predictable and easy to test.

**FR7 – Store fulfilment results**

The Fulfilment Service shall store the processing result in its H2 database.

The result shall include:

- event ID
- order ID
- fulfilment status
- reason for rejection, when applicable
- processed timestamp

**FR8 – Publish the fulfilment outcome**

The Fulfilment Service shall publish either:

OrderFulfilledEvent

or:

OrderRejectedEvent

to the Kafka topic:

order-result

**FR9 – Consume the result event**

The Order Service shall consume result events from the order-result topic.

**FR10 – Update order status**

The Order Service shall update the order status to:

FULFILLED

or:

REJECTED

**FR11 – Retrieve an order**

The client shall be able to retrieve an order by its ID through a REST endpoint.

Example:

GET /api/orders/{orderId}

**FR12 – List orders**

The client shall be able to retrieve all orders.

Example:

GET /api/orders

---

**6. Non-functional requirements**

**NFR1 – Maintainability**

The system shall use clear package boundaries and separation of concerns between:

- controllers

- application services
- domain objects
- repositories
- Kafka producers
- Kafka consumers
- configuration
- exception handling

**NFR2 – Testability**

Business logic shall be testable independently of Kafka, the database and the web layer.

Dependencies shall be injected through constructors so they can be mocked during unit testing.

**NFR3 – Reliability**

The system shall handle consumer failures using retry behaviour and a dead-letter topic.

**NFR4 – Idempotency**

The Fulfilment Service shall detect duplicate events and prevent the same order from being processed more than once.

**NFR5 – Traceability**

Every event shall include:

eventId

correlationId

occurredAt

This allows an order to be traced across both services.

**NFR6 – Validation**

All external input shall be validated before it reaches the business logic.

**NFR7 – Error handling**

The REST API shall return clear and consistent error responses for:

- invalid requests
- missing orders
- unexpected failures

**NFR8 – Performance**

The REST API should respond quickly after storing the order and publishing the event, without waiting for fulfilment to finish.

**NFR9 – Scalability**

The services shall be loosely coupled so that Kafka consumers could be scaled independently in a larger deployment.

**NFR10 – Portability**

The project shall run locally using:

- Java 21
- Spring Boot

- Gradle
- Kafka through Docker
- H2 in-memory databases

**NFR11 – Observability**

The system shall expose Spring Boot Actuator endpoints and use meaningful application logs for order and event processing.

**NFR12 – Clean code**

The implementation shall use:

- meaningful names
- small focused methods
- single-responsibility classes
- interfaces where abstraction is valuable
- minimal duplication
- consistent exception handling

---

## 7. Assumptions

- Authentication is outside the initial project scope.
- Payment processing is not included.
- Inventory is represented by a simple fulfilment rule rather than a full stock-management system.

- H2 data is reset whenever the application restarts.
- Kafka runs locally through Docker.
- The client may initially be Postman or Swagger rather than a frontend.
- The project uses two Spring Boot services only.

---

**8. Constraints**

- The laptop has 8 GB RAM.
- Kafka must remain a single lightweight broker.
- PostgreSQL will not be installed.
- Kubernetes, service discovery and service mesh are outside scope.
- The system must remain small enough to complete within the assignment deadline.
- The design should prioritise clean code, testing, Spring Boot and EDA over infrastructure complexity.

**9. Out of scope**

To prevent the capstone becoming too large, exclude:

- user registration and login
- real payment gateways
- email or SMS delivery
- a React frontend

- cloud deployment
- Kubernetes
- advanced inventory management
- multiple Kafka brokers
- distributed tracing platforms
- production-grade database clustering

**10. Success criteria**

The capstone will be considered successful when this flow works end to end:

POST /api/orders

→ order stored as PENDING

→ OrderCreatedEvent published

→ Fulfilment Service consumes event

→ fulfilment result stored

→ OrderFulfilledEvent or OrderRejectedEvent published

→ Order Service consumes result

→ order status updated

→ GET /api/orders/{id} returns final status